

Parcours du code du frontend au backend

Guide de préparation à la présentation chez l'encadreur. Il suit les fichiers réellement présents dans PROJECT et explique qui appelle qui, quelles données circulent et où les résultats sont enregistrés. Le document complète le guide des écrans ; il ne décrit pas une architecture future comme si elle existait déjà.

Le dossier à ouvrir dans l'éditeur

C:\Users\USER\PC\Desktop\MEMOIRESSHT\PROJECT

Tous les chemins courts de ce PDF sont relatifs à ce dossier. Par exemple, frontend/src/App.tsx désigne le fichier App.tsx dans PROJECT/frontend/src. Les références de fichiers sont cliquables dans les lecteurs PDF qui autorisent l'ouverture de fichiers locaux. Sinon, ouvrir le fichier dans l'éditeur et utiliser Ctrl+G pour atteindre la ligne indiquée.

Repères vérifiés sur le code du 9 octobre 2026. Les numéros de ligne peuvent changer après une modification. Le nom de la fonction ou du composant reste un second repère. Certains composants JSX occupent de longues lignes : utiliser aussi Ctrl+F pour retrouver le bouton ou le texte d'écran.

Le parcours principal

N	Fichier à suivre	Rôle
1	frontend/src/main.tsx	Démarre React dans la page HTML.
2	frontend/src/App.tsx	Charge la session, les données et les écrans.
3	frontend/src/forms.tsx	Prépare et envoie la photographie.
4	frontend/src/api.ts	Effectue les appels HTTP vers /api.
5	backend/app/main.py	Valide la demande et crée les enregistrements.
6	backend/app/jobs.py	Exécute le traitement en arrière-plan.
7	backend/app/segmentation.py	Lance YOLO et écrit les masques.
8	frontend/src/AnalysisDetail.tsx	Affiche les résultats renvoyés par le serveur.

Une distinction à retenir

Le frontend ne lance pas directement le modèle .pt et ne se connecte pas directement à PostgreSQL. Il envoie des requêtes HTTP à FastAPI. Le backend contrôle les droits, écrit en base et appelle le moteur de segmentation. Le navigateur reçoit des objets JSON et des URLs de fichiers protégés.

Le démarrage du frontend et les échanges HTTP

frontend/index.html : 1 ligne

Point d'entrée HTML. Le conteneur root reçoit l'application React ; le script source démarre main.tsx. Le dossier dist contient ensuite le résultat de la compilation, pas les fichiers à modifier au quotidien.

frontend/src/main.tsx : ligne 7

Crée la racine React, affiche App dans React.StrictMode et importe les styles Leaflet, style.css et sidebar.css. C'est le premier fichier TypeScript exécuté côté interface.

frontend/src/api.ts : ligne 5

La fonction api<T> préfixe le chemin avec /api, appelle fetch et transmet les cookies de même origine. Elle interprète les erreurs du serveur via ApiError. json prépare une requête JSON ; dateLabel et numberLabel ne font que formater l'affichage.

```
5 export async function api<T>(path:string, options:RequestInit={}):Promise<T>
  {
6   const response=await fetch('/api'+path,{...options,credentials:'same-
  origin'});
7   if(!response.ok){
8     let message='Le serveur ne peut pas traiter cette demande.';
9     try{const data=await response.json();message=typeof
      data.detail==='string'?data.detail:'Vérifiez les champs du
      formulaire.';}catch{ /* Keep network-safe message. */ }
10    throw new ApiError(message,response.status);
11  }
12  return response.json();
13 }
```

Exemple : api("/workspace") devient une requête vers /api/workspace. Pour importer une image, api reçoit un FormData : on laisse le navigateur définir le multipart et sa frontière. Pour lancer une analyse, json("POST", {confidence}) prépare un corps JSON.

frontend/src/types.ts : ligne 7

Décrit les structures TypeScript : User, Observation, Analysis, Result, Instance, Review, Settings, ModelInfo, Workspace et Sample. Ces types aident à vérifier le code à la compilation ; ils ne remplacent pas la validation des demandes par le serveur.

frontend/vite.config.ts : ligne 4

Active React et Tailwind. En développement, le proxy envoie /api vers le backend local 8001. Dans la version compilée utilisée pour la présentation, FastAPI sert directement frontend/dist.

frontend/package.json : 6 lignes

Déclare les dépendances et les commandes dev, build et preview. npm run build exécute la vérification TypeScript puis produit les fichiers compilés avec Vite.

À expliquer

Le navigateur construit l'interface, puis api.ts sert de pont HTTP. Un type TypeScript est un contrat de code ; la protection réelle des entrées se trouve côté FastAPI et Pydantic.

App la navigation et les styles

frontend/src/App.tsx : ligne 17

Composant central : possède l'utilisateur, le workspace, les filtres, la page courante, les messages et l'ouverture de l'import. Les composants enfants reçoivent leurs données et des callbacks.

Repère dans App.tsx	Fonctionnement
15 - readRoute	Lit location.hash : #dashboard ou #observations/identifiant. Le fragment après # n'est pas une route HTTP envoyée au serveur.
21 - refresh	GET /api/workspace puis setWorkspace. En cas de 401, retire l'utilisateur de l'état de l'interface.
22 - premier effet	GET /api/status puis /api/auth/me si la base est initialisée.
25 - actualisation	Recharge le workspace toutes les 1,5 seconde lorsqu'une analyse est queued ou running.
28 - go et open	Change la page et ouvre un dossier ; open peut mémoriser une analyse historique précise.
30 - onCreated	Après import, recharge, ouvre le dossier et demande l'analyse si la case automatique est cochée.

Les tableaux de bord, listes d'observations, historique, vérifications, modèle et carte sont rendus conditionnellement dans App.tsx. Les filtres de date, quartier et recherche sont calculés dans le navigateur à partir du workspace. Le filtre de date s'applique à created_at, la date d'ajout.

frontend/src/Sidebar.tsx : ligne 9

Affiche les liens reçus via items, les groupes Navigation et Configuration, le compte et les icônes Lucide. onNavigate revient vers go dans App ; onToggle réduit la barre. Le composant gère lui-même le menu du compte et sa fermeture par clic extérieur ou Échap.

frontend/src/style.css : 21 lignes

Styles généraux : panneaux, formulaires, états, tableau de bord, résultat, carte et adaptation aux différentes tailles d'écran.

frontend/src/sidebar.css : 45 lignes

Styles de la nouvelle navigation : largeur 248 px, mode réduit 68 px, état actif, groupes et pied de compte. main.tsx importe ce fichier après style.css afin d'appliquer les règles spécifiques.

À expliquer

Les écrans partagent un état central. Cliquer sur un menu change le hash et le rendu React ; cela ne relance pas un entraînement ou un calcul YOLO.

forms les trois familles de formulaires

frontend/src/forms.tsx : 41 lignes

AuthScreen la connexion

frontend/src/forms.tsx : ligne 7

submit empêche l'envoi HTML classique avec preventDefault, lit les champs via FormData puis construit un objet. Selon initialized, il appelle POST /api/auth/login ou /api/auth/setup. En cas de succès, onLogin transmet l'utilisateur à App ; le backend a également posé le cookie de session.

UploadModal la photographie

frontend/src/forms.tsx : ligne 17

file contient le fichier choisi. Un effet crée une URL temporaire pour son aperçu puis la libère. exemples charge GET /api/samples. chooseSample récupère la photo exemple, la convertit en File et renseigne son titre et ses notes de provenance.

frontend/src/forms.tsx : ligne 27

submit vérifie qu'une photo est choisie et contrôle 15 Mo côté client. Le FormData contient les champs et le fichier image. La case analyze est retirée du corps envoyé : elle sert uniquement à décider ensuite de lancer une seconde requête d'analyse. Les coordonnées et la date vides sont retirées. POST /api/observations renvoie l'observation ; onCreated dans App poursuit le parcours.

SettingsPage les paramètres et les comptes

frontend/src/forms.tsx : ligne 34

Fonction interne	Route envoyée	Rôle
save	PUT /api/settings	Enregistre nom, ville et seuil par défaut, puis recharge le workspace.
password	PUT /api/auth/password	Envoie le mot de passe actuel et le nouveau ; vide le formulaire au succès.
addUser	POST /api/users	Crée un compte puis recharge GET /api/users pour la liste.
useEffect de chargement	GET /api/users	Charge les comptes uniquement pour un administrateur.

Les validations existent à deux endroits

Les attributs HTML required, minLength et les contrôles React aident l'utilisateur. Ils sont contournables par une requête manuelle. Le backend revalide les tailles, les champs et les permissions. Une case masquée ou un bouton absent ne constitue donc pas la protection principale.

Les composants de consultation et le dossier

frontend/src/components.tsx : 64 lignes

Composant et ligne	Fonction
Status : 7	Traduit none, queued, running, completed et failed en badges.
ReviewStatus : 12	Affiche la dernière décision humaine ; elle est distincte du statut technique.
Empty : 16	Affiche un état vide avec une icône et une explication.
Modal : 19	Ouvre un dialogue HTML et gère son annulation ou un clic extérieur.
SearchField : 24	Champ contrôlé : sa valeur et son callback appartiennent au parent.
ObservationTable : 27	Affiche les observations, appelle onOpen et gère les pages de dix lignes ; cinq en mode compact.
ObservationMap : 34	Crée Leaflet, dessine les positions, adapte les limites et ouvre un dossier depuis un point.

frontend/src/AnalysisDetail.tsx : ligne 7

Composant du dossier : reçoit row, user, le seuil, la disponibilité du modèle et les callbacks. analysisId sélectionne une analyse ; mode choisit original, overlay ou mask ; zoom et confidence sont des états locaux.

frontend/src/AnalysisDetail.tsx : ligne 14

run appelle POST /api/observations/{id}/analyses, sélectionne l'identifiant renvoyé, revient à Original puis demande au parent de recharger les données. La requête crée une tâche ; elle ne contient pas le résultat final.

frontend/src/AnalysisDetail.tsx : ligne 15

review appelle POST /api/analyses/{id}/reviews avec decision et comment. onChange recharge le workspace. Le formulaire de vérification est disponible pour administrateur et vérificateur.

frontend/src/AnalysisDetail.tsx : ligne 16

Le champ image choisit row.image_url, analysis.overlay_url ou analysis.mask_url. Le navigateur télécharge ces fichiers via des routes protégées. Le même composant rend le nombre d'instances, la couverture, la durée, le tableau des objets et les empreintes. Les exports utilisent des liens de téléchargement directs.

Les premiers fichiers du backend

backend/app/config.py : ligne 5

Calcule ROOT et DATA, charge .env, lit DATABASE_URL, JOB_MODE et REDIS_URL, et définit MAX_BYTES et MAX_PIXELS. Les dossiers nécessaires sont créés. Les secrets de .env ne doivent pas être montrés pendant la présentation.

backend/app/main.py : ligne 36

Crée l'application FastAPI, les routes HTTP et les modèles Pydantic. lifespan vérifie l'accès au schéma et appelle jobs.recover au démarrage. browser_security contrôle l'origine des mutations et ajoute des en-têtes de protection.

backend/app/main.py : ligne 156

workspace charge les comptes, vérifications, analyses et observations. review_dict, analysis_dict et observation_dict transforment les objets de base en réponses JSON et fabriquent les URLs de fichiers. settings_dict fournit les paramètres. Le frontend reçoit ainsi un objet Workspace complet.

backend/app/auth.py : ligne 24

Regroupe le hachage, la comparaison de mot de passe, la lecture de session, les rôles et la création de cookie. current_user est injecté dans les routes via Depends ; require_role limite les actions.

Fonction et ligne dans auth.py	Ce qu'elle vérifie ou produit
hash_password : 11	Sel aléatoire et hachage scrypt, jamais stockage du mot de passe en clair.
check_password : 16	Recalcule le hachage et compare avec hmac.compare_digest.
current_user : 24	Cookie waste_session, empreinte en base, expiration et compte actif.
require_role : 37	Refuse une action avec HTTP 403 si le rôle n'est pas autorisé.
start_session : 41	Jeton aléatoire, empreinte en base, cookie HttpOnly SameSite strict valable douze heures.

POST /api/auth/login vérifie le compte et limite les tentatives. POST /api/auth/setup n'est autorisé que pour une base sans compte et crée l'administrateur. GET /api/auth/me renvoie l'utilisateur actuel. POST /api/auth/logout retire sa session. Les contrôles d'authentification ne sont pas exécutés dans le moteur YOLO : ils précèdent les actions métier.

backend/app/__init__.py : 1 ligne

Fichier d'identification du paquet Python app. Il ne contient pas le moteur d'analyse.

Le parcours exact de l'importation

Le clic jusqu'à la réponse HTTP 201

N	Repère à ouvrir	Données et opération
1	frontend/src/forms.tsx : 27	Le FormData porte image, title, neighborhood, location, notes et les champs facultatifs connus.
2	frontend/src/api.ts : 5	fetch envoie le corps multipart à /api/observations avec la session.
3	backend/app/main.py : 184	FastAPI lit UploadFile et Form, puis impose un rôle administrateur ou agent.
4	backend/app/main.py : 193	Contrôle de taille, image décodable, format JPEG PNG WebP et maximum de 20 millions de pixels.
5	backend/app/main.py : 207	GPS éventuel, orientation EXIF et conversion RGB.
6	backend/app/main.py : 214	Écriture JPEG normalisée, original .source et calcul des empreintes SHA256.
7	backend/app/main.py : 216	Création de la ligne observations en transaction ; observation_dict prépare la réponse.
8	frontend/src/App.tsx : 30	Réception du JSON, recharge du registre, ouverture du dossier et éventuelle demande d'analyse.

Les destinations de stockage

data/originals/{observation_id}.jpg contient la version normalisée. data/originals/{observation_id}.source conserve les octets importés. PostgreSQL conserve les champs de contexte, dimensions, identifiant du créateur et empreintes. Une erreur de transaction supprime les deux fichiers créés pour cet import.

Ce que la requête ne fait pas

Le backend ne reçoit pas le choix analyser : le frontend l'a retiré du FormData. L'importation et l'analyse sont deux demandes distinctes. Le serveur peut donc avoir enregistré une photo même si la demande d'analyse suivante échoue. L'interface signale ce cas et permet de relancer depuis le dossier.

Un identifiant n'est pas le titre

Le backend crée un UUID pour chaque observation. Les chemins de données utilisent cet identifiant, pas le nom saisi par l'utilisateur. L'analyse possède son propre UUID. C'est ce qui permet de conserver plusieurs traitements pour une même photographie sans écraser les résultats précédents.

La file de traitement et le moteur YOLO

backend/app/main.py : ligne 247

analyze contrôle le rôle, la disponibilité du modèle et l'existence de l'observation. Il refuse une autre analyse pending sur la même photo et limite la file à vingt traitements. Une ligne Analysis avec status queued est créée et validée en base, puis jobs.submit est appelé. La réponse HTTP 202 signifie tâche acceptée, pas calcul terminé.

backend/app/jobs.py : ligne 12

En mode local, submit utilise un ThreadPoolExecutor de un worker. run_analysis passe queued à running, appelle predict puis enregistre completed et result_json, ou failed en cas d'erreur. recover reprend les tâches encore queued et marque les traitements running interrompus comme failed.

```

12 def run_analysis(key):
13     with Session.begin() as db:
14         row = db.get(Analysis, key)
15         if not row or row.status != 'queued':
16             return
17         row.status = 'running'
18         observation_id, confidence = row.observation_id, row.confidence
19     try:
20         result = segmentation.predict(DATA / 'originals' /
f'{observation_id}.jpg', confidence, DATA / 'results' / key)
21         with Session.begin() as db:
22             row = db.get(Analysis, key)
23             row.status, row.finished_at = 'completed', now()
24             row.result_json = json.dumps(result, ensure_ascii=False)
25     except Exception:
26         logger.exception('Analysis failed %s', key)
27         with Session.begin() as db:
28             row = db.get(Analysis, key)

```

backend/app/segmentation.py : ligne 33

model_config lit active.json et vérifie le chemin des poids. model_status fournit la configuration visible. load_model vérifie SHA256 et classes, puis charge une seule fois YOLO dans un verrou.

backend/app/segmentation.py : ligne 56

predict lit la photo avec OpenCV, appelle model.predict avec conf, imsz=960, device=cpu et retina_masks=True. Pour chaque instance, il recueille classe, confiance, boîte et masque. mask_union forme une image binaire en conservant les trous et sans compter deux fois les pixels superposés.

Le moteur compose overlay.jpg, écrit mask.png et result.json, et calcule la couverture en pixels. Les contours et les couleurs facilitent la visualisation ; ils ne prouvent pas une bonne classification. La durée mesurée commence après load_model et ne correspond pas au temps total d'attente utilisateur.

backend/app/worker.py : ligne 10

Variante Celery : analyze_task appelle le même run_analysis via un broker Redis. Ce fichier est utilisé seulement si JOB_MODE vaut celery. La démonstration Windows utilise la variante locale ; la configuration Celery n'a pas été exécutée sur ce PC.

Le retour vers l'écran et les exports

La progression est récupérée par actualisation

frontend/src/App.tsx : ligne 25

Après le POST, App détecte une analyse queued ou running et relance GET /api/workspace toutes les 1,5 seconde. Il n'y a pas de WebSocket dans cette version. jobs.py met à jour la base ; workspace et analysis_dict relisent les résultats ; setWorkspace provoque le nouveau rendu React.

backend/app/main.py : ligne 68

analysis_dict décode result_json et prépare overlay_url et mask_url. AnalysisDetail utilise ces URLs pour afficher la superposition et le masque. Changer de vue ne recalcule pas YOLO ; cela change le fichier téléchargé et affiché.

Route HTTP	Fonction dans main.py	Utilisation
GET /api/observations/{key}/image	image_file : 238	Photo normalisée, session requise.
GET /api/analyses/{key}	analysis : 274	Consultation d'un traitement.
GET /api/analyses/{key}/files/{kind}	result_file : 279	kind = overlay, mask ou json ; analyse terminée requise.
POST /api/analyses/{key}/reviews	review : 293	Ajout d'une décision, pas modification du masque.
GET /api/observations/{key}/export	export_observation : 302	Dossier JSON avec analyses et vérifications.
GET /api/export/csv	export_all : 308	Registre CSV complet.
GET /api/export/geojson	export_all : 308	Positions réellement renseignées.

Un journal de décisions séparé

frontend/src/AnalysisDetail.tsx : ligne 15

Le formulaire transmet accepted, rejected ou uncertain et un commentaire. main.py exige un rôle admin ou reviewer et une analyse completed. Une nouvelle ligne Review est ajoutée. Les anciennes décisions restent conservées ; latest_review est la dernière. Les compteurs À vérifier utilisent cette dernière décision sur la dernière analyse de chaque observation.

Les exports sont des liens directs vers /api ; leur téléchargement n'emprunte pas obligatoirement api.ts. Ils restent protégés par current_user. Les filtres visuels sont locaux : les exports CSV et GeoJSON portent sur le registre entier.

Localisation modèle et configuration

Le trajet des coordonnées

backend/app/main.py : ligne 169

Les champs latitude et longitude proviennent de forms.tsx. create_observation impose la paire complète, donne priorité à la saisie manuelle et lit le GPS EXIF si la paire est absente. Observation conserve latitude, longitude et coordinate_source. workspace les renvoie ; App filtre les photos localisées et les passe à ObservationMap.

frontend/src/components.tsx : ligne 34

Le premier effet crée Leaflet et le fond OpenStreetMap. Le second vide et recrée les points quand rows change, puis utilise fitBounds. Un clic sur Ouvrir l'observation appelle le callback du parent. Les coordonnées localisent la photo ; YOLO ne fournit pas une adresse depuis les masques.

frontend/src/components.tsx : ligne 42

La couche de tuiles possède referrerPolicy strict-origin-when-cross-origin. Elle transmet l'origine locale requise par OpenStreetMap malgré la politique globale same-origin du serveur. Les autres échanges de l'application conservent leurs protections. Le fond nécessite Internet.

La page Modèle est surtout alimentée par le workspace

backend/app/segmentation.py : ligne 20

models/active.json : 14 lignes

Déclare les poids actifs, nom, provenance, révision, classes, empreinte et limites. models/waste-yolo11s-seg.pt est le fichier de poids exécuté, pas un fichier JavaScript. Si models/evaluation.json existe et correspond au SHA du modèle actif, model_status expose ses métriques locales.

Le rendu Modèle YOLO se trouve dans App.tsx. Les métriques externes TACO sont explicitement séparées des métriques locales. Le mode détaillé par découpes et l'édition manuelle de masques sont des propositions, pas des branches de code déjà actives.

Paramètres comptes et exemples côté serveur

Route	Repère dans main.py
GET et PUT /api/settings	get_settings : 331 ; update_settings : 341
PUT /api/auth/password	password : 355
GET et POST /api/users	users : 367 ; create_user : 376
GET /api/samples et /api/samples/{key}	samples : 389 ; sample_file : 394
GET /api/status et /api/health	status : 92 ; health : 97

La base les migrations et les fichiers

backend/app/database.py : ligne 9

Crée engine à partir de DATABASE_URL et Session pour les transactions. Les classes SQLAlchemy décrivent les tables ; les créer dans le fichier Python ne remplace pas la migration du schéma.

Classe et ligne	Table	Données importantes
User : 19	users	Nom, login, hachage, rôle et actif.
LoginSession : 29	login_sessions	Empreinte de jeton, utilisateur et expiration.
Observation : 35	observations	Photo référencée par UUID, contexte, GPS, dimensions et empreintes.
Analysis : 53	analyses	Observation liée, auteur, état, seuil, dates, erreur et result_json.
Review : 65	reviews	Analyse liée, décision, commentaire, vérificateur et date.
Setting : 74	settings	Paramètres sous forme clé et valeur.

Les relations sont assurées par des clés étrangères : une Analysis référence une Observation ; une Review référence une Analysis. Les créateurs et vérificateurs référencent des User. L'observation possède plusieurs analyses ; une analyse possède plusieurs vérifications. Ces liens expliquent pourquoi les journaux peuvent être conservés sans écraser les traitements.

backend/migrations/env.py : ligne 7

Configure Alembic avec Base.metadata et la connexion de l'application, puis exécute les migrations.

backend/migrations/versions/0001_initial.py : 77 lignes

Première migration enregistrée : décrit les tables et index de départ. start.ps1 applique alembic upgrade head avant de lancer FastAPI.

backend/alembic.ini : 3 lignes

Configuration d'Alembic et chemin du dossier de migrations. Elle est passée à la commande avec -c.

Les chemins de données

Chemin dans PROJECT	Contenu
data/originals/{observation_id}.jpg	Photo normalisée utilisée par OpenCV et YOLO.
data/originals/{observation_id}.source	Fichier initial importé, conservé pour traçabilité.
data/results/{analysis_id}/overlay.jpg	Superposition colorée.
data/results/{analysis_id}/mask.png	Union des masques en pixels.
data/results/{analysis_id}/result.json	Résultat détaillé du traitement.
runtime/postgres et backups	Données du cluster local et archives de sauvegarde.

DATA est configurable par WASTE_DATA_DIR. Une sauvegarde complète doit conserver PostgreSQL et les fichiers image.

Lancement et outils complémentaires

LANCER.cmd : 3 lignes

Point de lancement par double-clic. Appelle le script PowerShell start.ps1.

start.ps1 : 12 lignes

Se place dans PROJECT, prépare PostgreSQL, exécute Alembic, vérifie frontend/dist puis lance Unicorn app.main:app avec --app-dir backend et le port 8001.

install.ps1 : 10 lignes

Prépare l'environnement Python, les dépendances, le frontend compilé, les poids et les exemples. Sert à l'installation, pas à chaque clic d'analyse.

Script dans scripts	Rôle
prepare_database.py	Démarre ou prépare le cluster PostgreSQL local du projet et sa connexion.
download_model.py	Récupère la révision de modèle prévue et les exemples, avec contrôle des poids.
convert_coco.py	Convertit des annotations polygonales COCO en données YOLO segmentées et partitions.
dataset_tools.py	Vérifie les images, annotations, coordonnées et doublons exacts entre partitions.
train.py	Commande d'entraînement à lancer après préparation d'un corpus annoté.
register_model.py	Contrôle et enregistre de nouveaux poids de segmentation dans le manifeste actif.
evaluate.py	Évalue des poids sur une partition de test et écrit un rapport traçable.
backup.py	Archive dump PostgreSQL et fichiers nécessaires à la restauration.
reset_password.py	Outil local de réinitialisation de mot de passe avec invalidation des sessions.

Configuration et tests

.env contient les valeurs locales sensibles ; .env.example présente les variables attendues sans fournir les secrets. backend/requirements.txt et frontend/package-lock.json documentent les dépendances. Dockerfile et compose.yaml décrivent une variante de déploiement avec PostgreSQL, Redis et worker, non exécutée sur le PC de démonstration.

backend/app/main.py : ligne 404

À la fin de main.py, /assets sert les fichiers compilés et spa renvoie frontend/dist/index.html. Une route API inconnue reste une erreur 404, au lieu de renvoyer silencieusement l'interface.

tests contient les tests automatisés de logique et d'API. docs/qa contient les contrôles navigateur et les preuves techniques ; ces scripts ne sont pas le moteur utilisé à chaque analyse. Le corpus de Goma, son entraînement et son évaluation scientifique restent à réaliser.

Ordre de lecture et repères pour expliquer le code

Lire la chaîne dans cet ordre

N	Fichier et repère	Question à laquelle il répond
1	frontend/src/main.tsx : 7	Où React démarre-t-il ?
2	frontend/src/App.tsx : 21	D'où viennent les données des écrans ?
3	frontend/src/forms.tsx : 27	Comment le formulaire envoie-t-il la photo ?
4	frontend/src/api.ts : 5	Comment la demande devient-elle une requête HTTP ?
5	backend/app/main.py : 184	Où le serveur contrôle-t-il et enregistre-t-il l'image ?
6	backend/app/main.py : 247	Où l'analyse est-elle mise en file ?
7	backend/app/jobs.py : 12	Qui exécute le calcul et change les états ?
8	backend/app/segmentation.py : 56	Où YOLO produit-il les masques ?
9	backend/app/main.py : 68	Comment les résultats redeviennent-ils du JSON ?
10	frontend/src/AnalysisDetail.tsx : 16	Comment Original Segmentation Masque sont-ils affichés ?

Où chercher si quelque chose ne fonctionne pas

Connexion : forms.tsx, route login et auth.py. Import : submit de UploadModal et create_observation.

Analyse bloquée : analyze, jobs.py et journal serveur. Masque absent : segmentation.py, fichier dans

data/results et route result_file. Carte sans point : coordonnées dans Observation puis ObservationMap.

Carte bloquée : requêtes des tuiles et referrerPolicy. Chiffre inattendu : filtres et calculs dans App.tsx.

Ce que je peux dire à l'encadreur

« J'ai séparé l'affichage du calcul. React envoie une photo à FastAPI ; le serveur la contrôle et l'enregistre. Une tâche lance YOLO puis OpenCV prépare les masques. PostgreSQL garde l'état et les informations, et l'interface récupère le résultat. Une vérification humaine est ensuite ajoutée séparément. »

« Les fichiers que je présente correspondent au parcours testé. Le modèle est externe ; l'intégration fonctionne. Je dois encore constituer les annotations de terrain et mesurer les performances sur un jeu de test indépendant. »

Utiliser les repères sans modifier par hasard

Ouvrir les fichiers pour expliquer leur rôle. Ne pas modifier .env, les poids ou la base pour une simple démonstration. Après un changement frontend, une compilation est nécessaire pour actualiser dist. Après un changement Python sans mode reload, le serveur doit être redémarré. Les fichiers de ce guide sont des repères de lecture, pas des instructions de déploiement.